



DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ N'GUESSAN
Nom d'usage ▶
Prénom ▶ SERGE ANGLESY
Adresse ▶ 4 PLACE DU VAL 45100 ORLEANS

Titre professionnel visé

Administrateur système devOps

MODALITE D'ACCES :

- ☐ Validation des Acquis de l'expérience (VAE)
- ☒ Parcours de formation

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE. Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- ▶ pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- ▶ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▶ une déclaration sur l'honneur à compléter et à signer ;
- ▶ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▶ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Automatiser le déploiement d'une infrastructure dans le cloud

4.

- Provisionnement de l'infrastructure AWS avec Terraform.....4.
- Sécurisation de l'accès sans SSH via AWS SSM6.

Déployer une application en continu

7.

- Mise en place du CI/CD avec github Actions7.
- Conteneurisation de l'application avec Docker9.

Superviser les services déployés

10.

- Déploiement de la stack Prometheus / Grafana 11.
- Configuration des alertes et health checks..... 12.

Déclaration sur l'honneur

14.

Activité-type n°1 – Automatiser le déploiement d'une infrastructure dans le cloud

Exemple n°1 ► Provisionnement de l'infrastructure AWS avec Terraform

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet StockLive (application de gestion de stock multi-magasins), nous avons conçu et provisionné l'ensemble de l'infrastructure cloud AWS en utilisant Terraform.

Tâches réalisées :

- Création d'un VPC avec sous-réseaux publics (10.0.1.0/24, 10.0.2.0/24) pour l'ALB et l'EC2, et sous-réseaux privés (10.0.3.0/24, 10.0.4.0/24) pour la base de données RDS.
- Configuration de l'Internet Gateway, des tables de routage et des associations de sous-réseaux.
- Déploiement d'une instance EC2 (t3.micro, Ubuntu 22.04 LTS) avec un user-data d'installation automatique de Docker.
- Création d'une base de données Amazon RDS MySQL 8.0 (db.t3.micro) en Multi-AZ avec sauvegardes automatiques activées.
- Mise en place d'un Application Load Balancer (ALB) avec groupe cible et health checks sur le port 8000.
- Gestion du state Terraform à distance via un bucket S3 chiffré avec verrouillage DynamoDB pour éviter les conflits d'exécution simultanée.
- Génération dynamique du mot de passe RDS via la ressource Terraform 'random_password' pour ne jamais stocker de secrets en clair.

2. Précisez les moyens

Terraform (IaC) – fichiers vpc.tf, compute.tf, database.tf, load_balancer.tf, security.tf, iam.tf, backend.tf, outputs.tf, variables.tf

- AWS CLI configuré avec clés d'accès IAM
- Amazon Web Services : VPC, EC2, RDS MySQL, ALB, S3, DynamoDB, IAM
- Région AWS : eu-west-3 (Paris)
- Git / GitHub pour la gestion du code d'infrastructure
- Visual Studio Code avec extension Terraform

3. Avec qui avez-vous

Travail réalisé de manière autonome dans le cadre de la formation. Échanges réguliers avec le formateur pour validation des choix d'architecture (segmentation réseau, choix des types d'instances, stratégie de backup RDS).

4. Contexte

Nom de l'entreprise, organisme ou association ▶ Ecole-IT

Chantier, atelier, service ▶ StockLive

Période d'exercice ▶ Du 22 Mars au 28 juin

5. Informations complémentaires

Le choix d'une architecture Multi-AZ pour RDS garantit la haute disponibilité en cas de défaillance d'une zone de disponibilité AWS. Le backend S3 + DynamoDB pour le state Terraform est une pratique de production indispensable pour le travail en équipe

Exemple n°2 ► Sécurisation de l'accès aux instances EC2 via AWS SSM

(sans SSH)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Nous avons mis en place un accès sécurisé aux instances EC2 sans exposition du port SSH (22), en remplaçant complètement l'accès traditionnel par SSH par AWS Systems Manager (SSM) Session Manager.

Tâches réalisées :

- Désactivation totale du port 22 dans les Security Groups EC2 (aucune règle inbound SSH).
- Création d'un rôle IAM pour l'instance EC2 avec la policy managée 'AmazonSSMManagedInstanceCore'.
- Attachement du profil d'instance IAM à l'EC2 lors du provisionnement Terraform.
- Configuration de la commande de déploiement applicatif via 'aws ssm send-command' depuis le pipeline CI/CD GitHub Actions.
- Mise en place d'une boucle d'attente (polling) pour vérifier que l'instance est enregistrée et 'Online' dans SSM avant d'exécuter le déploiement (jusqu'à 40 tentatives).
- Audit des sessions : toutes les commandes exécutées via SSM sont journalisées dans AWS CloudTrail, offrant une traçabilité complète.

2. Précisez les moyens utilisés :

- AWS Systems Manager (SSM) – Session Manager et Run Command
- Terraform – fichiers iam.tf, security.tf, compute.tf
- GitHub Actions – workflow ci.yml (étape 'Deploy via SSM')
- AWS CLI (commandes ssm describe-instance-information, ssm send-command)
- AWS IAM – rôle EC2 avec policy AmazonSSMManagedInstanceCore

3. Avec qui avez-vous travaillé ?

Travail autonome. Validation de l'approche avec le formateur qui a confirmé que l'élimination du port SSH représente une bonne pratique de sécurité ('Zero Trust' et principe du moindre privilège).

4. Contexte

Nom de l'entreprise, organisme ou association ► Ecole-IT

Chantier, atelier, service	► StockLive	
Période d'exercice	► Du 22 Mars	au 28 juin

Activité-type n°2 – Déployer une application en continu

Exemple n°1 ► Mise en place du pipeline CI/CD avec Github Actions

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Nous avons conçu et implémenté un pipeline d'intégration et de déploiement continu (CI/CD) complet avec GitHub Actions pour l'application StockLive.

Étapes du pipeline :

1. TESTS : Démarrage d'un service MySQL 8.0 dans l'environnement CI, installation des dépendances Python, exécution des tests de migration de base de données (test_migration.py).
2. BUILD DOCKER : Construction de l'image Docker multi-stage (builder + runtime) taguée 'stocklive-api:latest'.
3. SCAN DE SÉCURITÉ : Analyse de l'image avec Trivy pour détecter les vulnérabilités HIGH et CRITICAL ; le pipeline échoue automatiquement si des vulnérabilités critiques sont détectées.
4. INFRASTRUCTURE : Exécution de 'terraform init' et 'terraform apply' pour provisionner ou mettre à jour l'infrastructure AWS.
5. DÉPLOIEMENT : Envoi de commandes via AWS SSM pour puller le code, générer le fichier .env avec les credentials RDS, builder et démarrer les services Docker Compose, attendre le health check de l'API (30s timeout, 30 tentatives).
6. DIAGNOSTICS : Capture des logs de conteneur pour faciliter le débogage.

2. Précisez les moyens utilisés

- GitHub Actions – fichier .github/workflows/ci.yml
- Docker + Docker Hub (build et push d'images)
- Trivy (scanner de sécurité de conteneurs)
- Terraform (provisionnement infrastructure)
- AWS CLI + SSM (déploiement sur EC2)
- Python 3.11 + pytest (tests automatisés)
- Secrets GitHub Actions (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, DB credentials)

3. Avec qui avez-vous travaillé

Travail autonome avec revue du formateur sur la stratégie de pipeline (ordre des étapes, gestion des secrets, conditions de déclenchement sur push main / PR).

4. Contexte

Chantier, atelier, service

► StockLive

Période d'exercice

► Du 22 Mars au 28 juin

5. Informations complémentaires (facultatif)

Le pipeline est déclenché sur push vers main (deploy complet) et sur les Pull Requests (tests + build uniquement, sans deploy). La séparation permet de valider le code avant la mise en production tout en évitant des déploiements non intentionnels.

Exemple n°2 ► Conteneurisation de l'application FastAPI avec Docker

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Nous avons conteneurisé l'application StockLive (API FastAPI + base de données MySQL) en utilisant Docker et Docker Compose.

Tâches réalisées :

- Rédaction d'un Dockerfile multi-stage :
 - Stage 'builder' : python:3.11-slim + installation des dépendances de compilation (gcc, libmariadb-dev) + pip install dans /root/.local
 - Stage 'runtime' : python:3.11-slim + copie des dépendances + création d'un utilisateur non-root 'appuser' pour la sécurité + healthcheck sur /health toutes les 30s
- Configuration du Docker Compose local (docker-compose.yml) :
 - Service 'db' : MySQL 8.0 avec healthcheck, volume persistant 'db_data', scripts d'initialisation dans ./init_db, non exposé sur l'hôte
 - Service 'api' : FastAPI exposé sur le port 8000, dépend du service db avec condition 'service_healthy', chargement du fichier .env
 - Réseau bridge isolé 'stocklive-network'
- Optimisation de la taille d'image via le multi-stage build (image finale ~150 MB vs ~400 MB sans multi-stage).

2. Précisez les moyens utilisés

- Docker Engine
- Docker Compose v2
- Dockerfile (multi-stage build)
- python:3.11-slim comme image de base (légère et sécurisée)
- mysql:8.0 pour le service de base de données
- SQLAlchemy ORM pour l'accès à la BDD
- FastAPI + Uvicorn (serveur ASGI)

3. Avec qui avez-vous travaillé ?

Travail autonome. La stratégie multi-stage a été discutée avec le formateur qui a validé l'approche et recommandé l'ajout d'un utilisateur non-root pour réduire la surface d'attaque.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ Ecole-IT

Chantier, atelier, service

▶ StockLive

Période d'exercice

▶ Du 22 Mars

au 28 juin

Activité-type n°3 – Superviser les services déployés

Exemple n°1 ► Déploiement de la stack de supervision Prometheus / Grafana

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet StockLive (application de gestion de stock multi-magasins), nous avons conçu et provisionné l'ensemble de l'infrastructure cloud AWS en utilisant Terraform.

Tâches réalisées :

- Création d'un VPC avec sous-réseaux publics (10.0.1.0/24, 10.0.2.0/24) pour l'ALB et l'EC2, et sous-réseaux privés (10.0.3.0/24, 10.0.4.0/24) pour la base de données RDS.
- Configuration de l'Internet Gateway, des tables de routage et des associations de sous-réseaux.
- Déploiement d'une instance EC2 (t3.micro, Ubuntu 22.04 LTS) avec un user-data d'installation automatique de Docker.
- Création d'une base de données Amazon RDS MySQL 8.0 (db.t3.micro) en Multi-AZ avec sauvegardes automatiques activées.
- Mise en place d'un Application Load Balancer (ALB) avec groupe cible et health checks sur le port 8000.
- Gestion du state Terraform à distance via un bucket S3 chiffré avec verrouillage DynamoDB pour éviter les conflits d'exécution simultanée.
- Génération dynamique du mot de passe RDS via la ressource Terraform 'random_password' pour ne jamais stocker de secrets en clair.

2. Précisez les moyens utilisés :

- Prometheus (collecte et stockage de métriques time-series)
- Grafana (visualisation et dashboards)
 - Node Exporter (métriques système Linux)
 - AlertManager (routage d'alertes)
 - prometheus-fastapi-instrumentator (instrumentation Python)
 - Docker Compose (monitoring/docker-compose.yml)
 - Discord Webhook pour les notifications d'alerte

3. Avec qui avez-vous travaillé ?

Travail autonome avec consultation de la documentation officielle Prometheus et Grafana.
Présentation de la stack au formateur avec démonstration des métriques collectées en temps réel.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ Ecole-IT

Chantier, atelier, service ▶ StockLive

Période d'exercice ▶ Du 22 Mars au 28 juin

4. Informations complémentaires

La stack de monitoring est séparée de la stack applicative via un Docker Compose dédié, ce qui permet de la déployer indépendamment et de monitorer plusieurs applications sans modifier leur configuration.

Exemple n°2 ► Configuration des alertes et health checks

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai configuré des règles d'alerte automatisées et des health checks à plusieurs niveaux pour garantir la disponibilité et la fiabilité de l'application.

Tâches réalisées :

- Rédaction des règles d'alerte Prometheus (alert_rules.yml) :
 - Alerte 'HighErrorRate' : se déclenche si le taux de réponses 5xx dépasse un seuil pendant plus de 5 minutes
 - Alerte 'ServiceDown' : détecte la défaillance d'un service conteneurisé
- Configuration d'AlertManager avec routage vers un webhook Discord pour notification immédiate de l'équipe.
- Mise en place des health checks Docker sur le conteneur API (curl <http://localhost:8000/health> toutes les 30s, timeout 10s, 3 retries).
- Configuration des health checks ALB AWS sur le chemin /health (intervalle 30s, seuil sain: 2, seuil malsain: 5).
- Implémentation de l'endpoint /health dans l'API FastAPI retournant le statut de la base de données et de l'application.
- Dans le pipeline CI/CD : vérification du health check après déploiement (boucle de 30 tentatives avec délai de 10s entre chaque) avant de considérer le déploiement comme réussi.

2. Précisez les moyens utilisés :

- AlertManager – fichier alertmanager.yml
- Prometheus – fichier alert_rules.yml
- Docker healthcheck (Dockerfile et docker-compose.yml)
- AWS ALB Health Check (target group, fichier load_balancer.tf)
- FastAPI endpoint /health (main.py)
- Discord Webhook pour les notifications
- GitHub Actions pour la vérification post-déploiement

3. Avec qui avez-vous travaillé ?

Travail autonome. Validation des seuils d'alerte avec le formateur pour s'assurer qu'ils sont pertinents (ni trop sensibles pour éviter les faux positifs, ni trop permissifs pour ne pas manquer d'incidents réels).

4. Contexte

Nom de l'entreprise, organisme ou association ▶ Ecole-IT

Chantier, atelier, service ▶ StockLive

Période d'exercice ▶ Du 22 Mars au 28 juin

Déclaration sur l'honneur

Je soussigné(e) [prénom et nom] ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteur(e) des réalisations jointes.

Fait à le
pour faire valoir ce que de droit.

Signature

